

COMP0008: Computer Architecture and Concurrency

Live Session (week 6)

Stefano Vissicchio

References: P&H chapters 4-5

From C to execution in a MIPS processor

- Consider the following code segment in C:
 $a = b + e;$
 $c = b + f;$
- Assume that variables' values are in memory and are addressable as offsets from \$t0
- How is the code executed within a pipelined MIPS processor?

From C to execution in a MIPS processor

- Consider the following code segment in C:
 $a = b + e;$
 $c = b + f;$
- Assume that variables' values are in memory and are addressable as offsets from \$t0
- How is the code executed within a pipelined MIPS processor?
- Towards the answer:
 - Translate the C code into MIPS instructions
 - Simulate execution of MIPS instructions within a pipelined processor

From C to execution in a MIPS processor

- Consider the following code segment in C:
 $a = b + e;$
 $c = b + f;$
- Assume that variables' values are in memory and are addressable as offsets from \$t0
- How is the code executed within a pipelined MIPS processor?
- Towards the answer:
 - Translate the C code into MIPS instructions
 - Simulate execution of MIPS instructions within a pipelined processor

From C code to MIPS instructions

- Let's start from the first line
$$a = b + e;$$
- In MIPS, we need to:
 - Load the values of the variables in two registers
 - Store the sum in a third register
 - “Save” the sum in main memory

From C code to MIPS instructions

- Let's start from the first line
 $a = b + e;$
- In MIPS, we need to:
 - Load the values of the variables in two registers
 - Store the sum in a third register
 - “Save” the sum in main memory
- Where are the values of the variables initially?

From C code to MIPS instructions

- Let's start from the first line
`a = b + e;`
- In MIPS, we need to:
 - Load the values of the variables in two registers
 - Store the sum in a third register
 - “Save” the sum in main memory
- Where are the values of the variables initially?
 - The exercise stated: “Assume that variables’ values are in memory and are addressable as offsets from \$t0”
 - So, b is in 0(\$t0), e is in 4(\$t0), f is in 8(\$t0)

From C code to MIPS instructions

- Let's start from the first line

`a = b + e;`

- The line is translated into

`lw $t1, 0($t0)`

`lw $t2, 4($t0)`

`add $t3, $t1, $t2`

`sw $t3, 12($t0)`

- A similar procedure applies to the second line

`c = b + f;`

From C code to MIPS instructions

- So for the code segment

`a = b + e;`

`c = b + f;`

- The MIPS code is:

`lw $t1, 0($t0)`

`lw $t2, 4($t0)`

`add $t3, $t1, $t2`

`sw $t3, 12($t0)`

`lw $t4, 8($t0)`

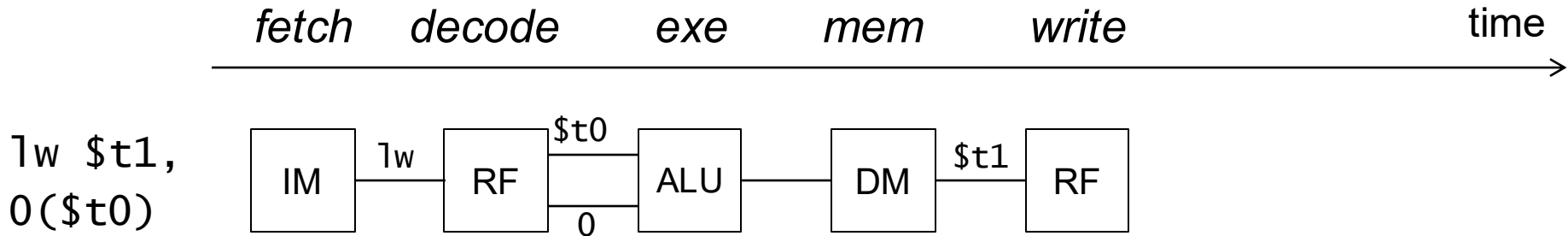
`add $t5, $t1, $t4`

`sw $t5, 16($t0)`

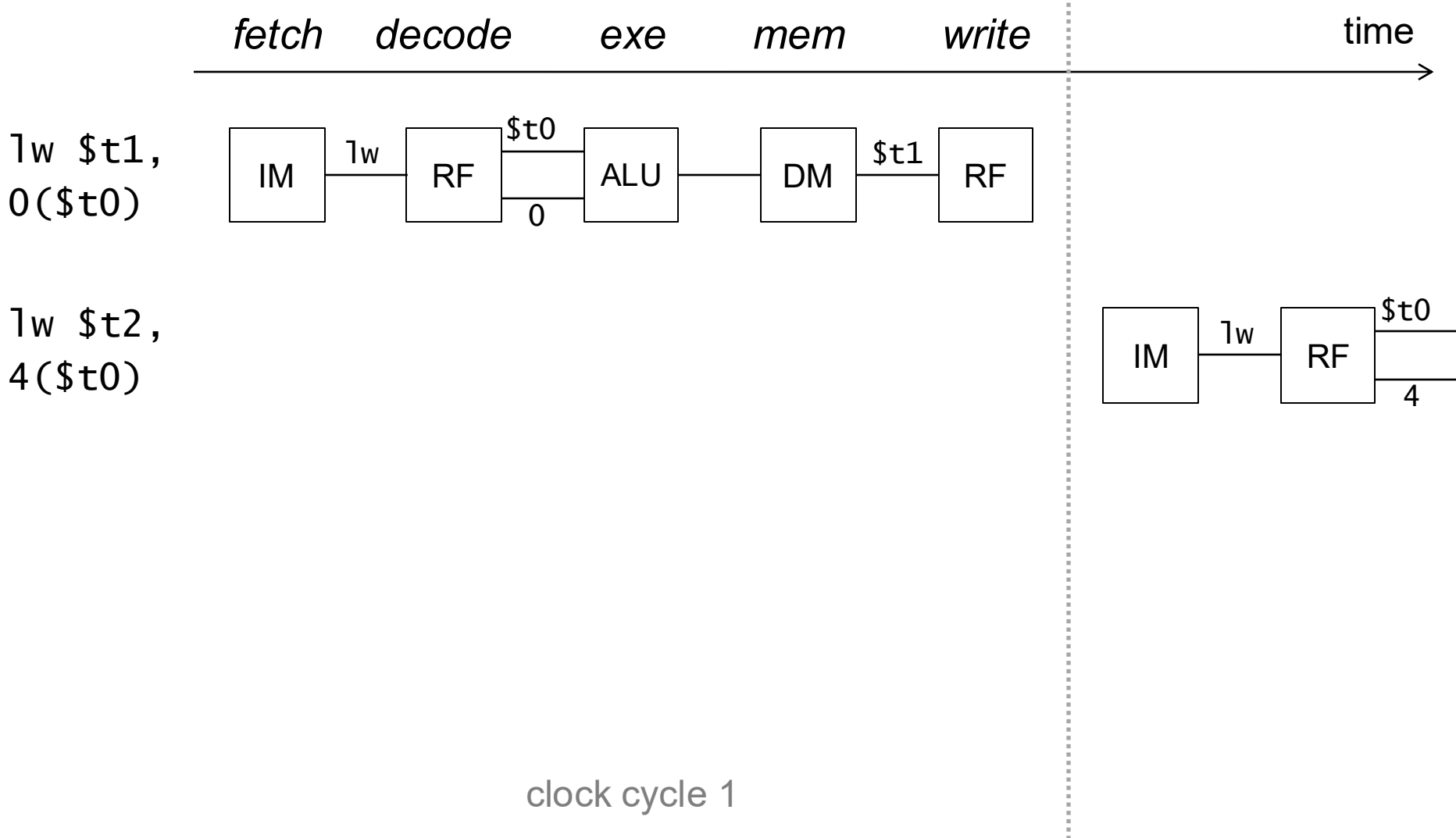
From C to execution in a MIPS processor

- Consider the following code segment in C:
 $a = b + e;$
 $c = b + f;$
- Assume that variables' values are in memory and are addressable as offsets from \$t0
- How is the code executed within a pipelined MIPS processor?
- Towards the answer:
 - Translate the C code into MIPS instructions
 - **Simulate execution of MIPS instructions within a pipelined processor**

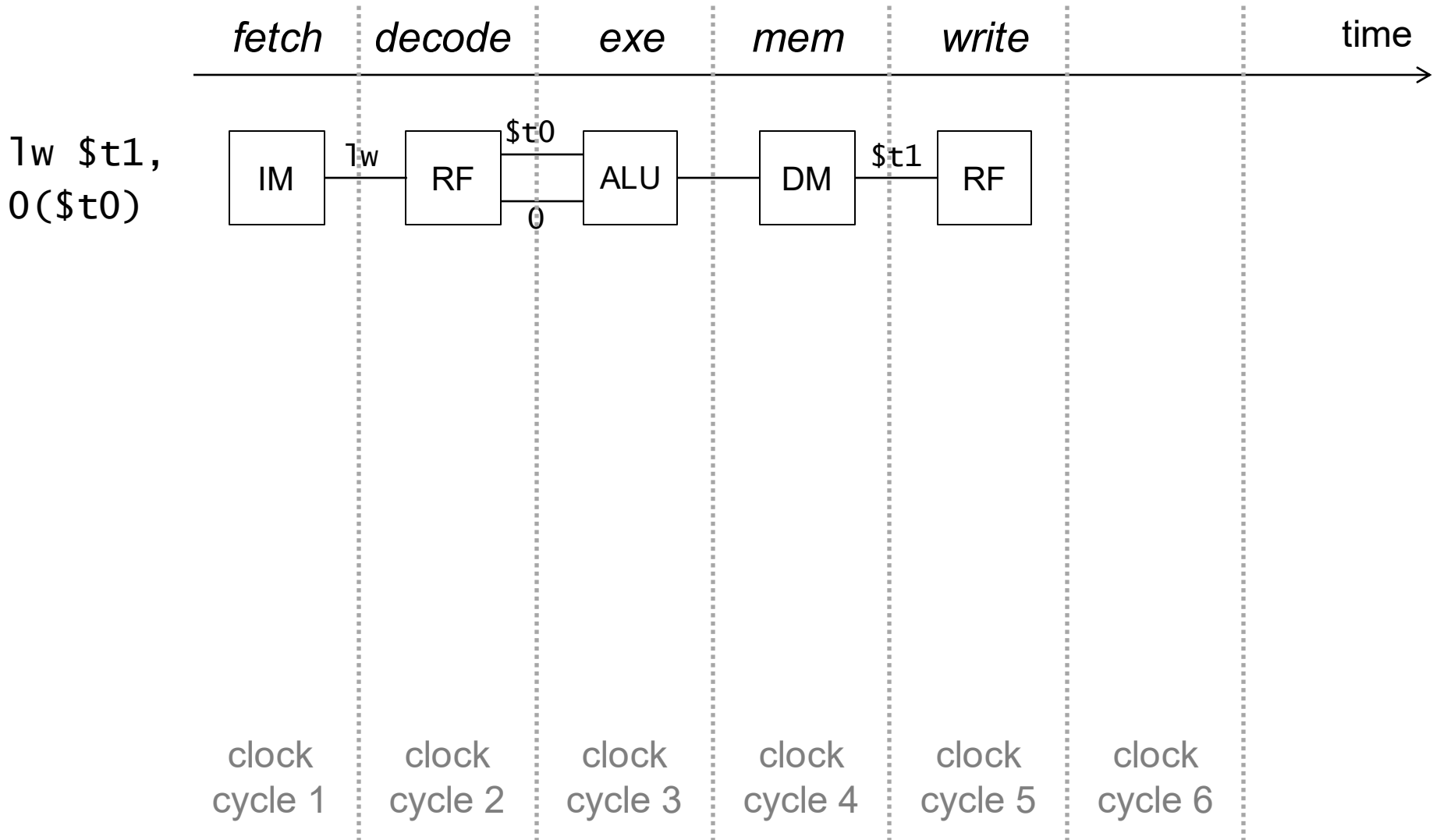
Executing one MIPS instruction



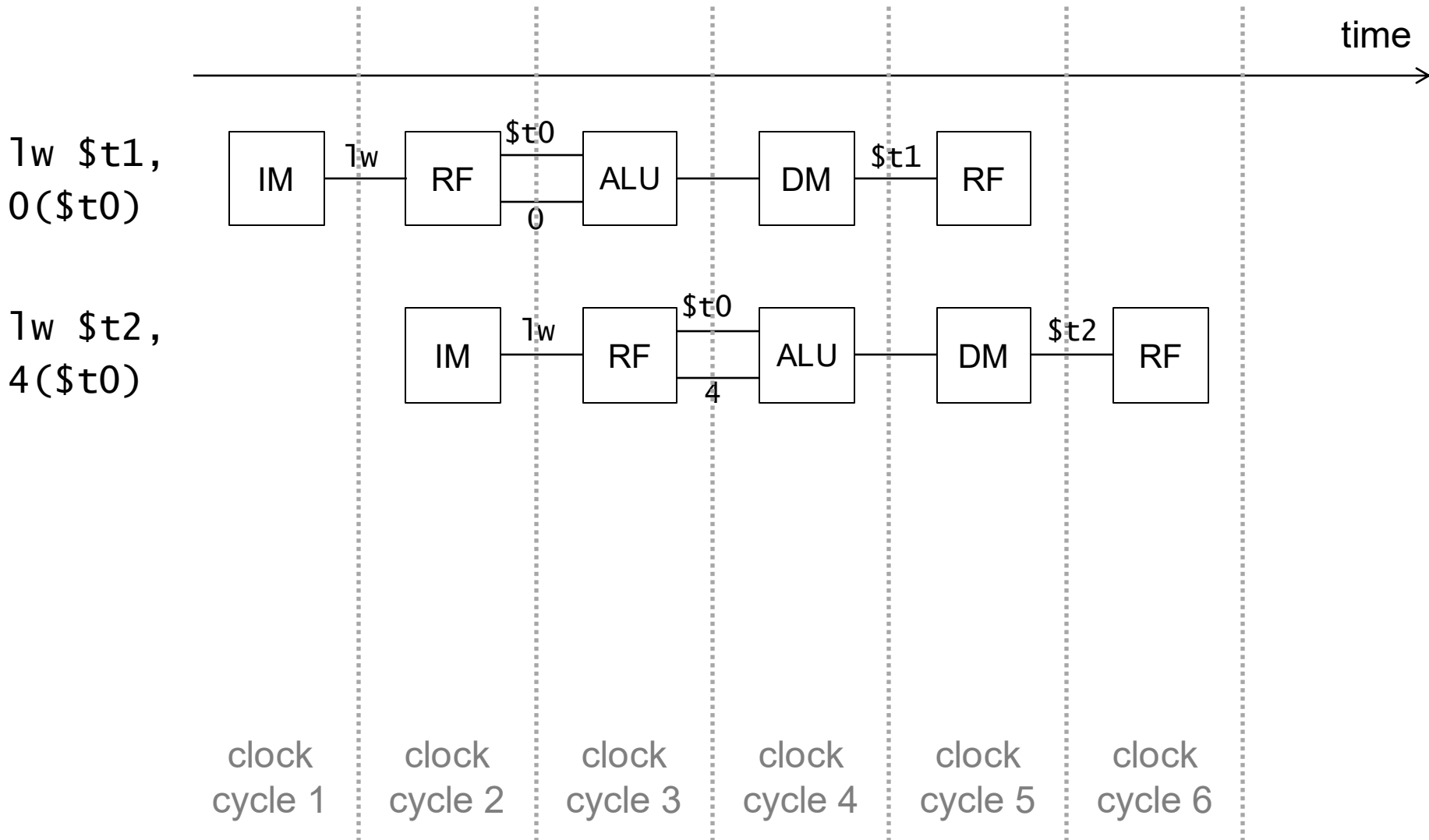
Executing MIPS instructions: no pipelining



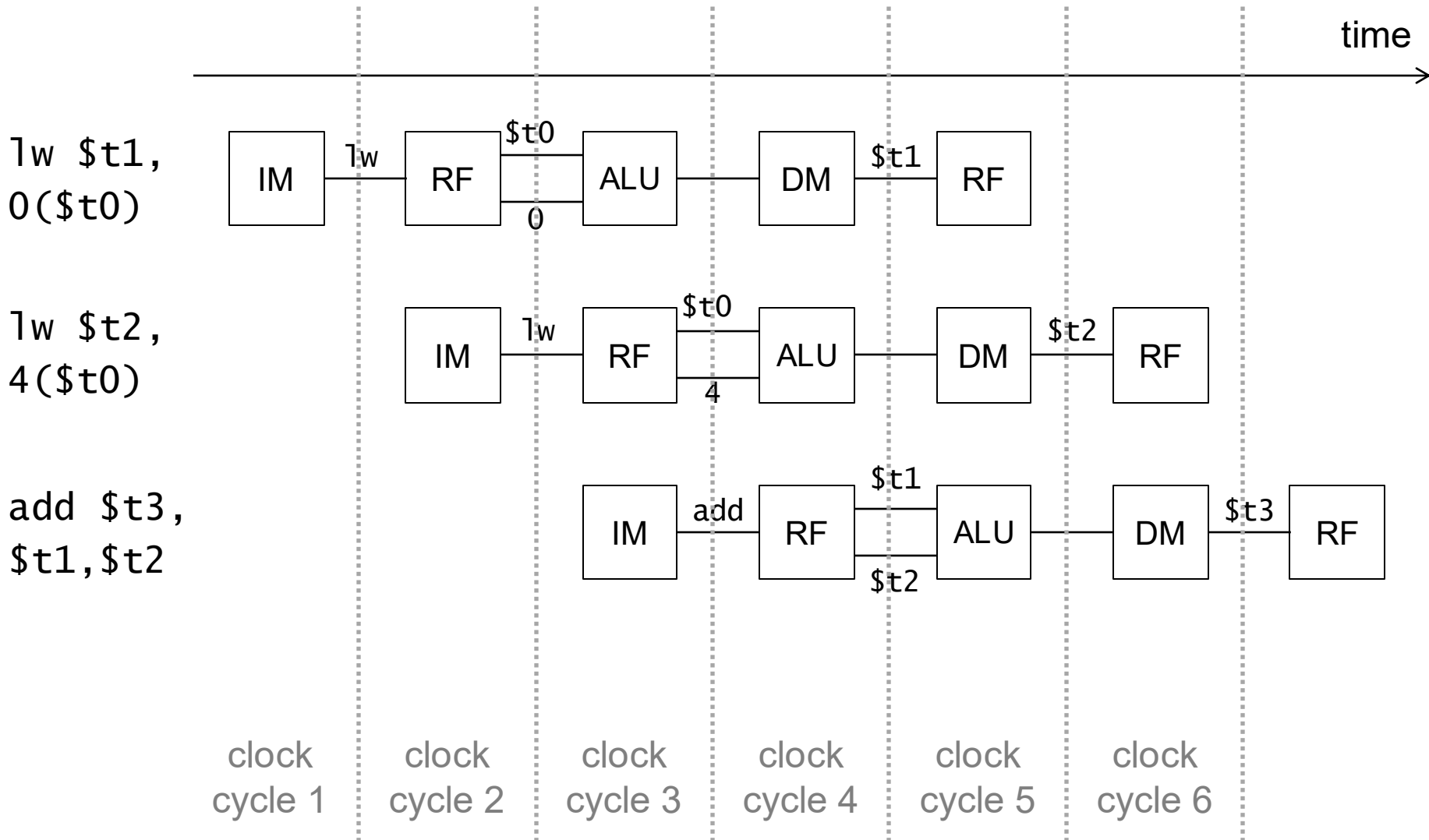
Executing MIPS instructions: pipelining



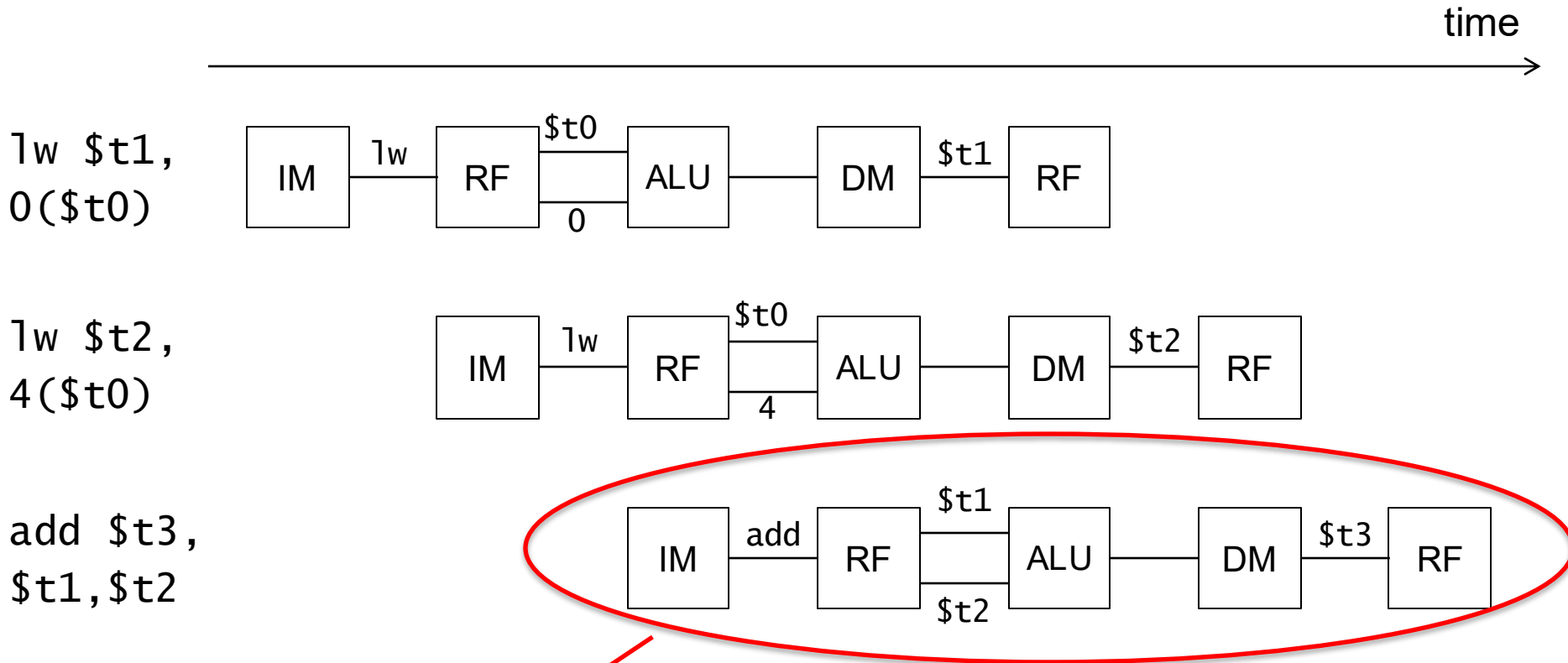
Executing MIPS instructions: pipelining



Executing MIPS instructions: pipelining

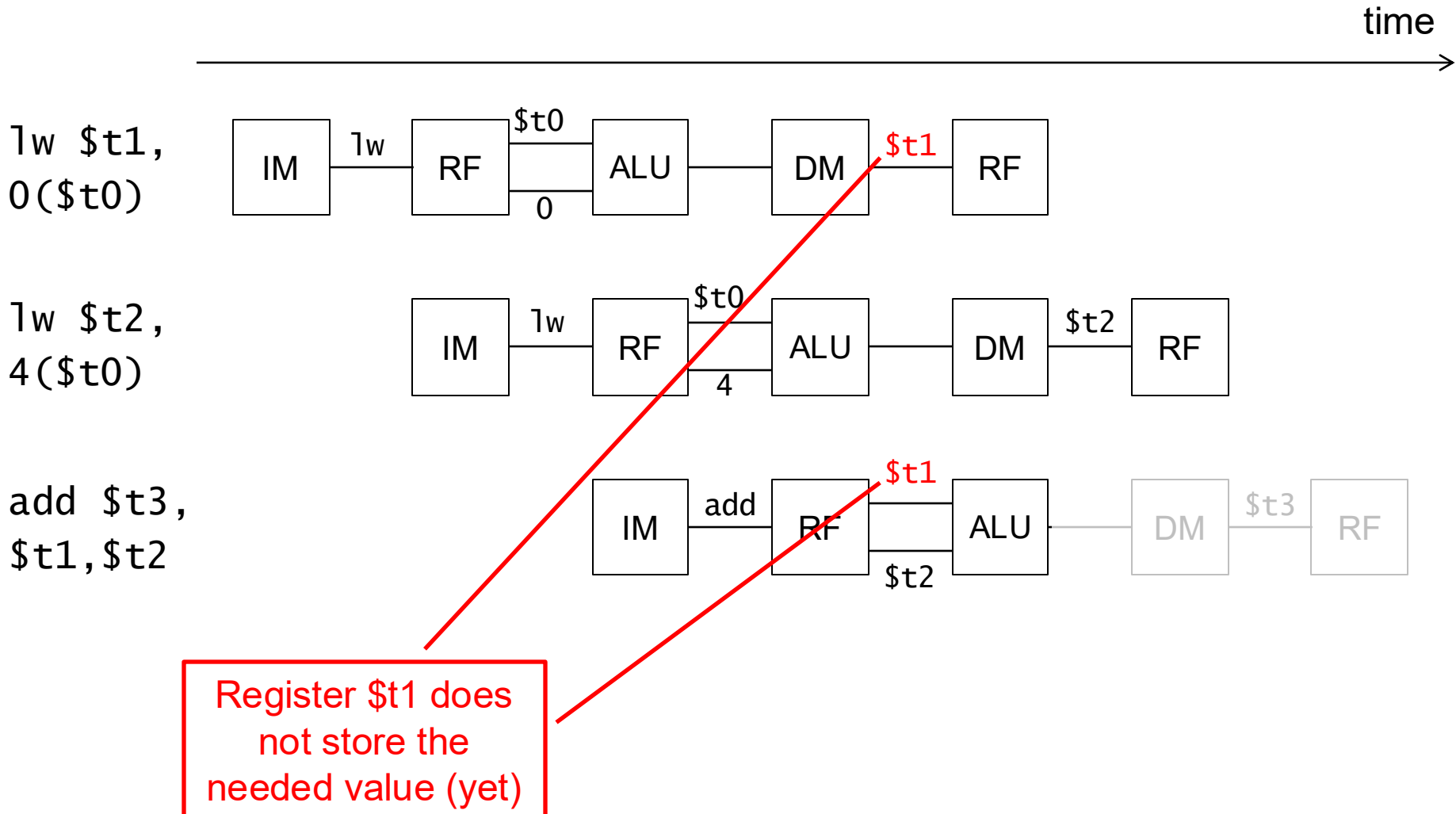


Executing MIPS instructions: pipelining

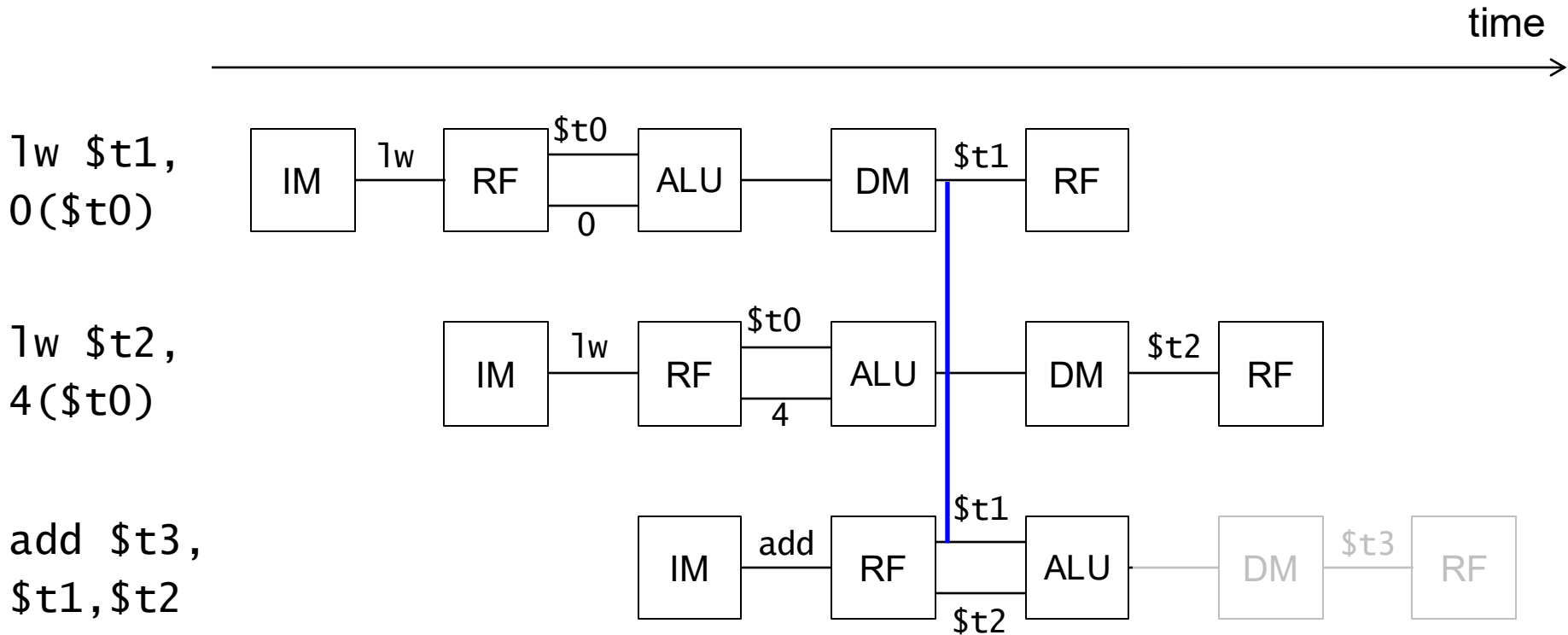


Can we execute this third MIPS instruction, though?

Executing MIPS instructions: data hazard

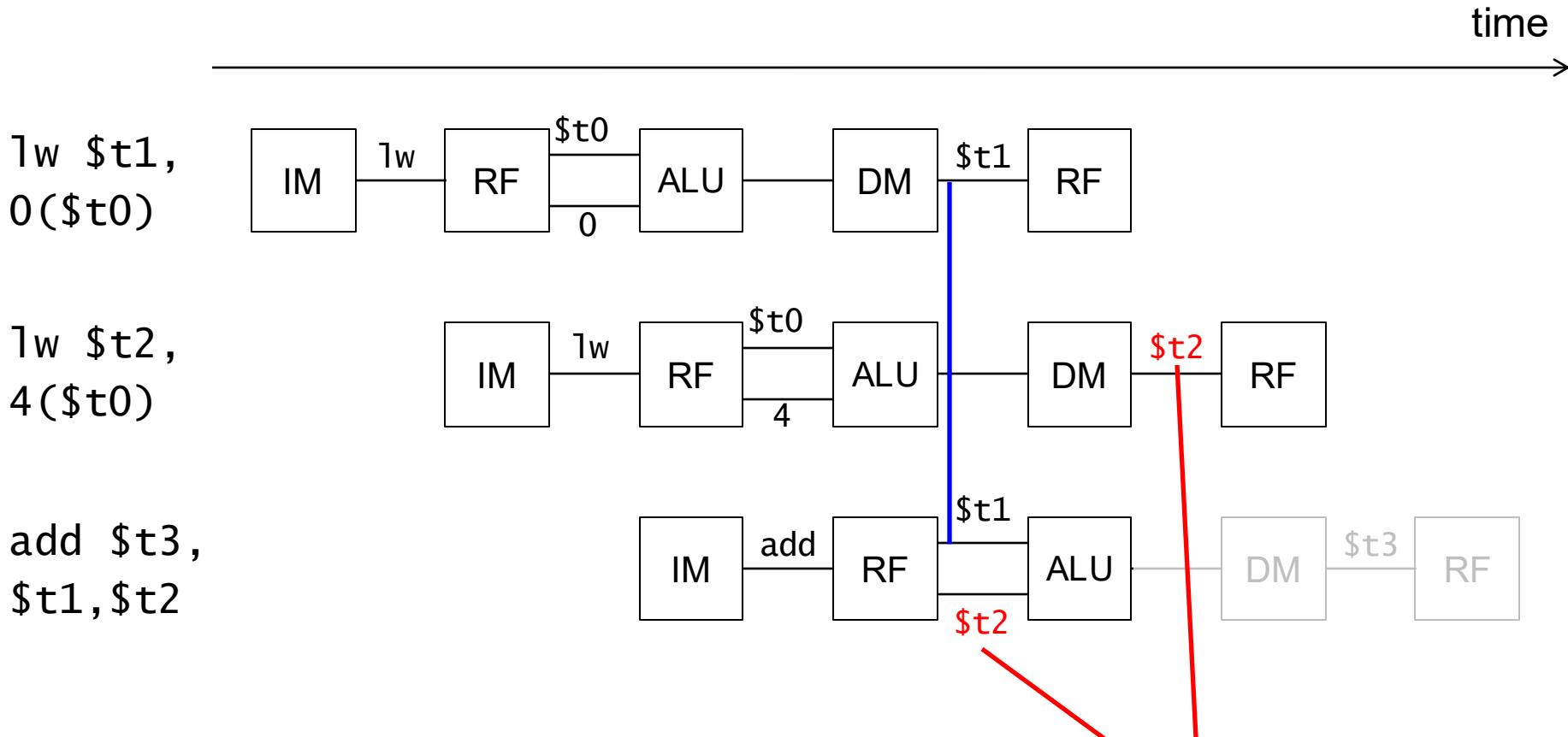


Executing MIPS instructions: forwarding



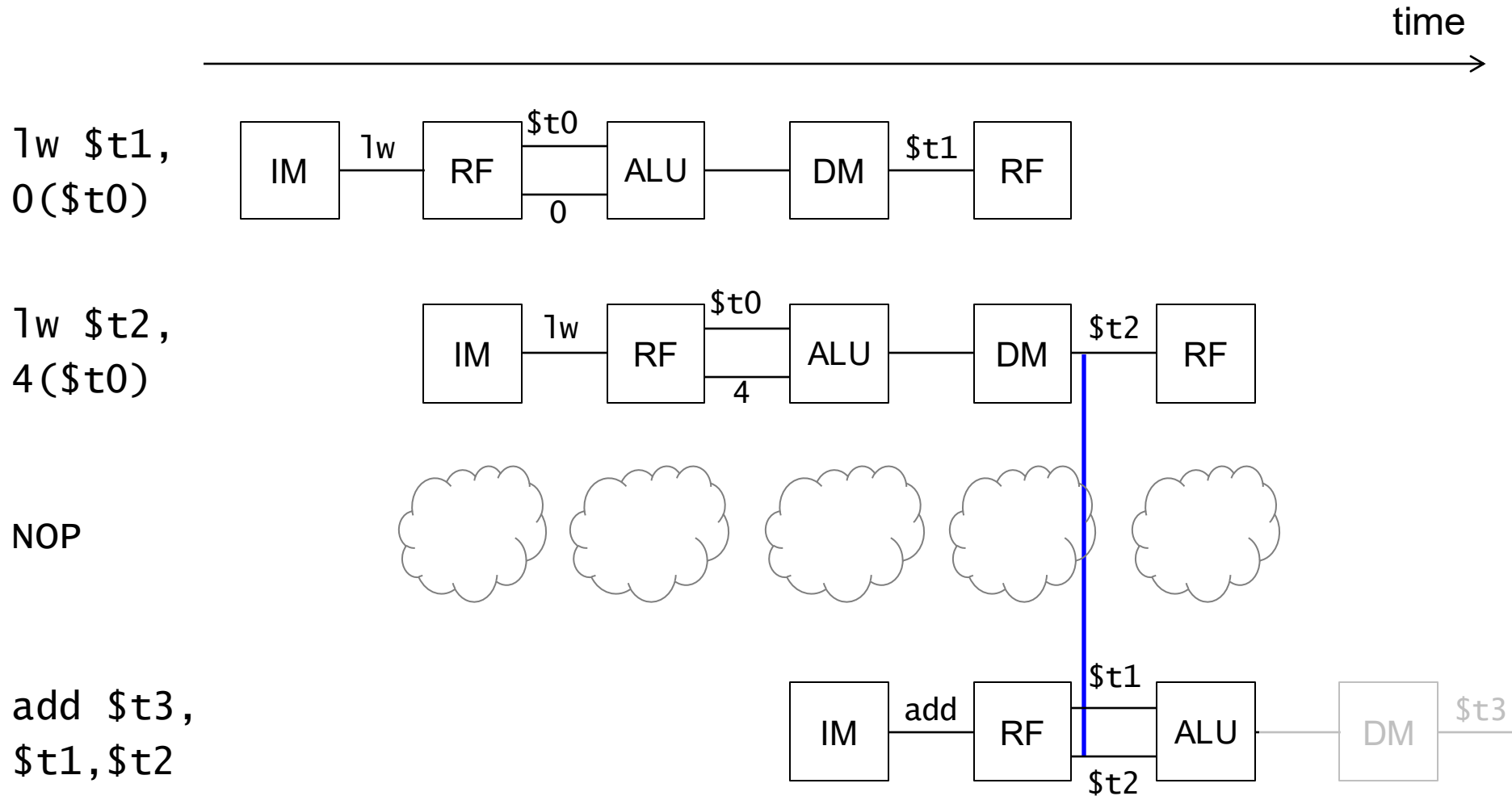
To solve this data hazard, we can use **forwarding** (assuming it is supported by the processor)

Executing MIPS instructions: data hazard #2



Register `$t2` does not store the needed value (yet) – and we cannot use forwarding this time

Executing MIPS instructions: stalling



Executing MIPS instructions: done?

- Actually executable sequence of MIPS instructions

```
lw    $t1, 0($t0)
lw    $t2, 4($t0)
nop
add    $t3, $t1, $t2
sw    $t3, 12($t0)
lw    $t4, 8($t0)
nop
add    $t5, $t1, $t4
sw    $t5, 16($t0)
```

Executing MIPS instructions: reordering

- Compilers and processors can improve performance by reordering instructions

```
lw    $t1, 0($t0)
```

```
lw    $t2, 4($t0)
```

```
nop
```

```
add   $t3, $t1, $t2
```

```
sw    $t3, 12($t0)
```

```
lw    $t4, 8($t0)
```

```
nop
```

```
add   $t5, $t1, $t4
```

```
sw    $t5, 16($t0)
```

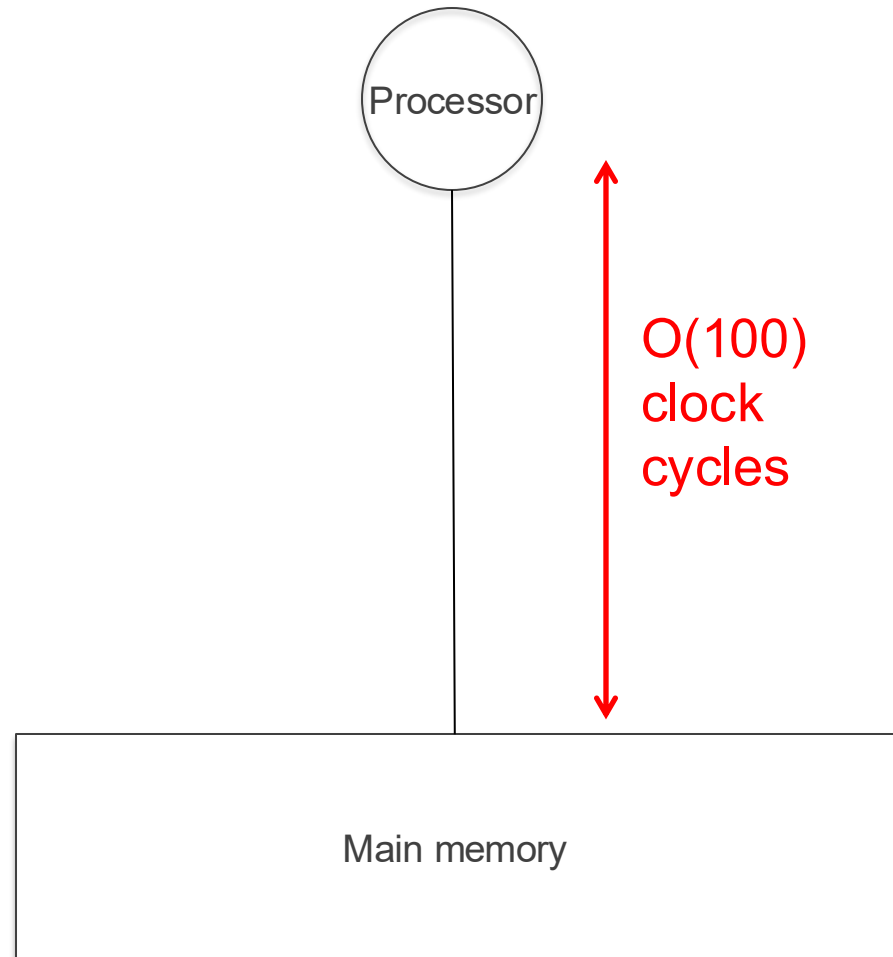


Executing MIPS instructions: fast?

- Pipelining increases instruction throughput
- Does this make execution fast?

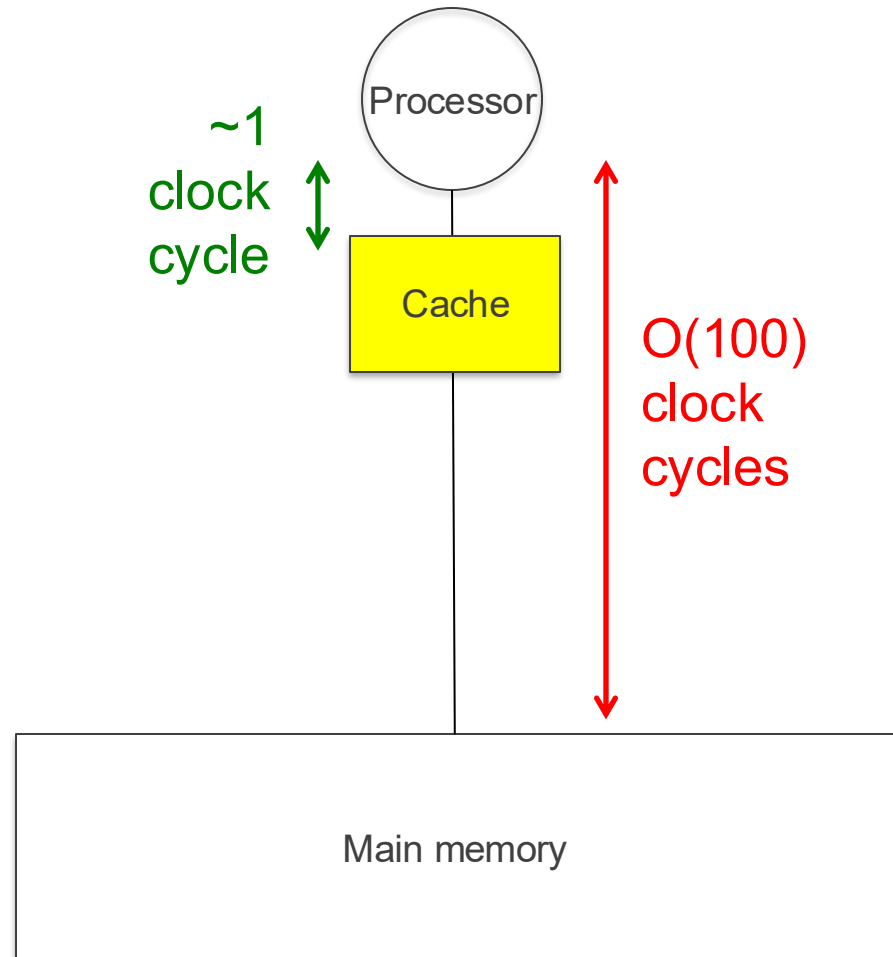
Interaction with memory

- Pipelining increases instruction throughput
- Does this make execution fast?
 - makes it *faster*, but...
- Now, the bottleneck is **access to memory**
 - e.g., for lw and sw instructions



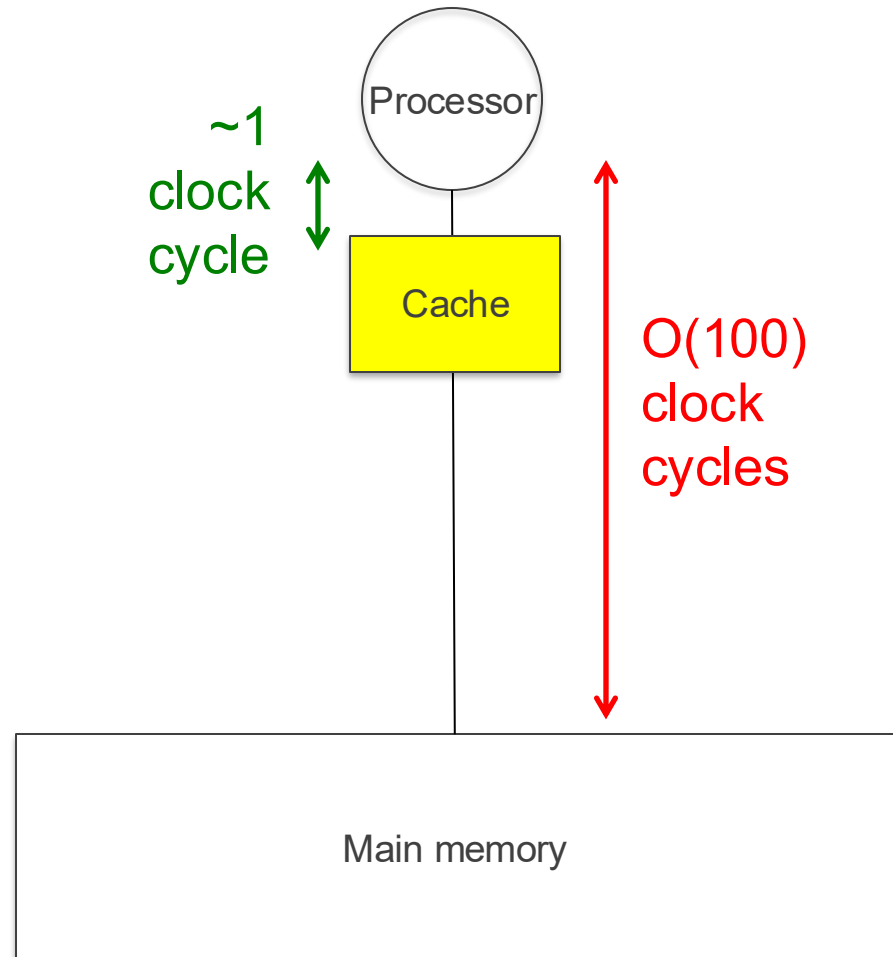
Interaction with memory

- Pipelining increases instruction throughput
- Does this make execution fast?
 - makes it *faster*, but...
- Now, the bottleneck is **access to memory**
 - e.g., for `lw` and `sw` instructions
- Workaround: **cache**
 - bring data closer to processors



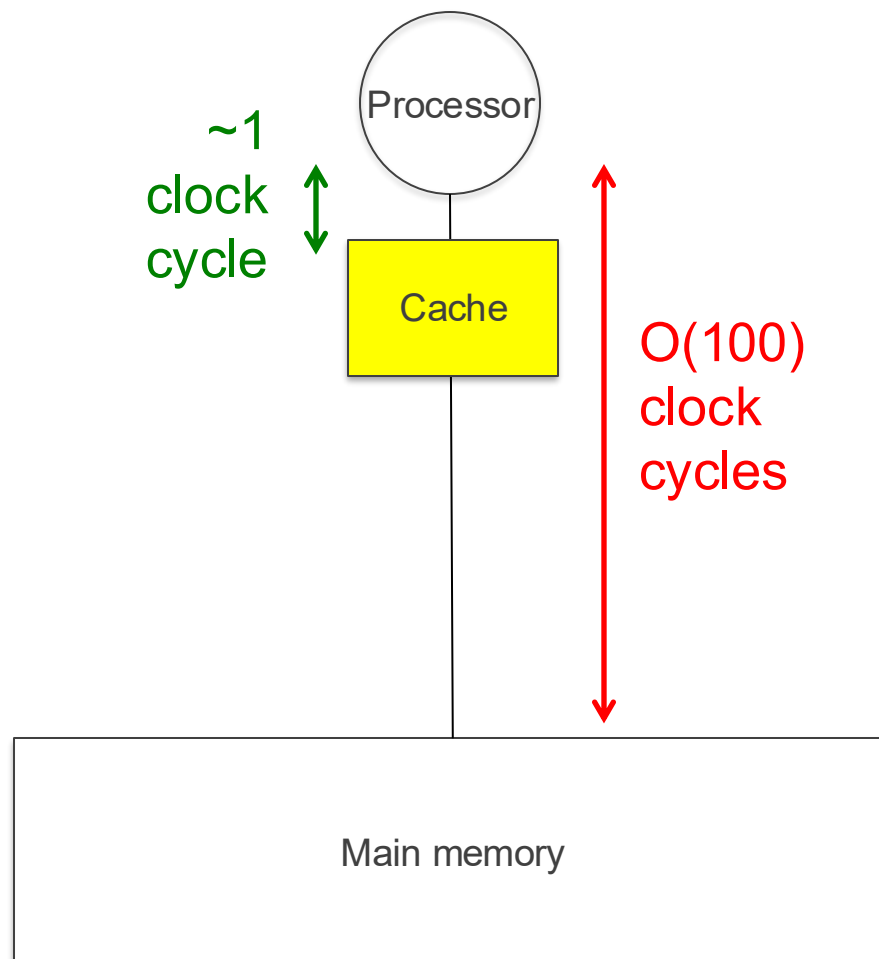
Interaction with memory: caches

- Fast to access, but **expensive** and **small**
 - Contains small subset of main memory
- Caching works well because of **locality**
 - **Temporal locality**: caches store copies of **recently used** data
 - **Spatial locality**: caches store copies of data **close to recently used one**



Interaction with memory: caches in action

- Suppose processor has to "load a word"
- We **check cache first**
 - **cache hit** → just return data in cache
 - **cache miss** → data loaded from memory and cache updated
- Cache **updates**:
 - **missed data**, for temporal locality
 - **close mem locations**, for spatial locality

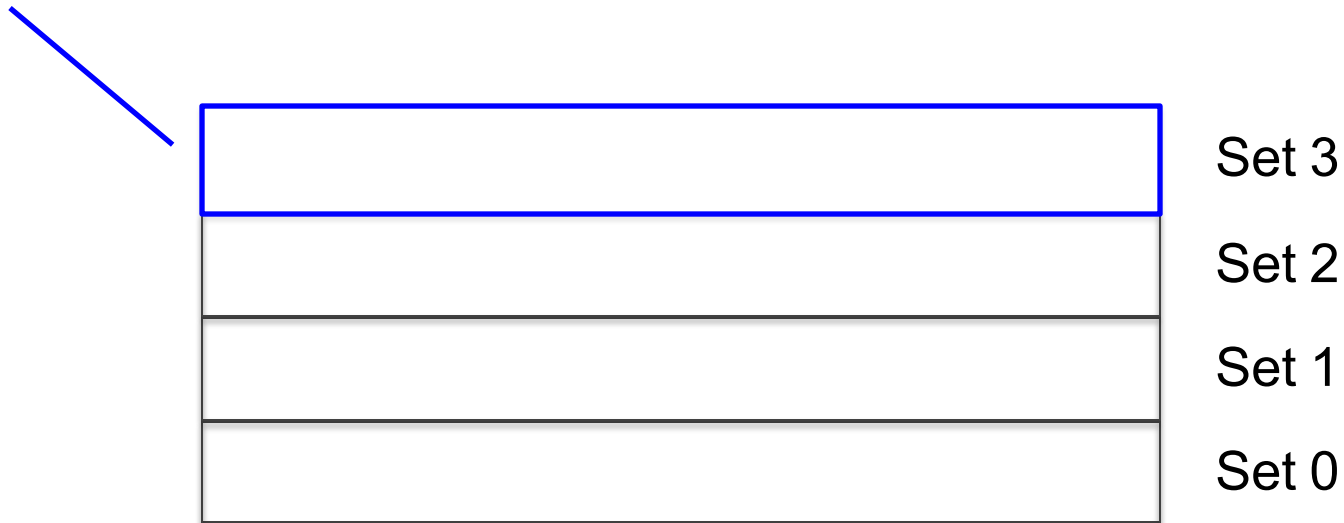


Interaction with memory: caches internals



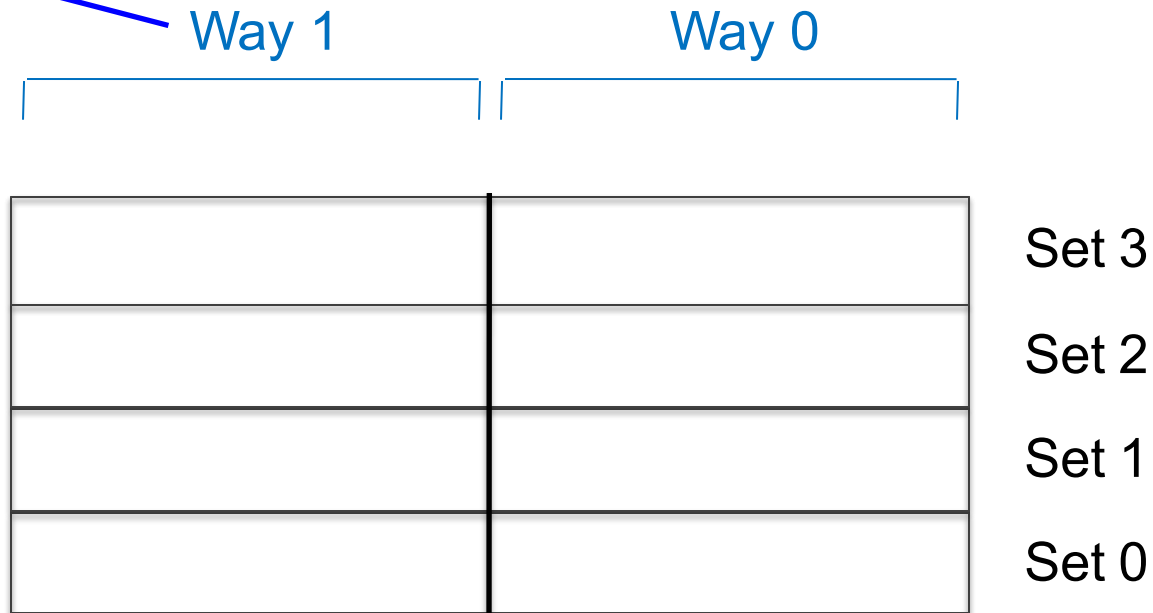
Interaction with memory: caches internals

Location where to store the content of *specific* addresses in main memory
(i.e., specific memory blocks)



Interaction with memory: caches internals

Each way
stores data
contiguous in
main memory



Interaction with memory: caches internals

Each way
stores data
contiguous in
main memory
(+ metadata)

Way 1			Way 0			
V	tag	data	V	tag	data	
						Set 3
						Set 2
						Set 1
						Set 0

Interaction with memory: caches internals

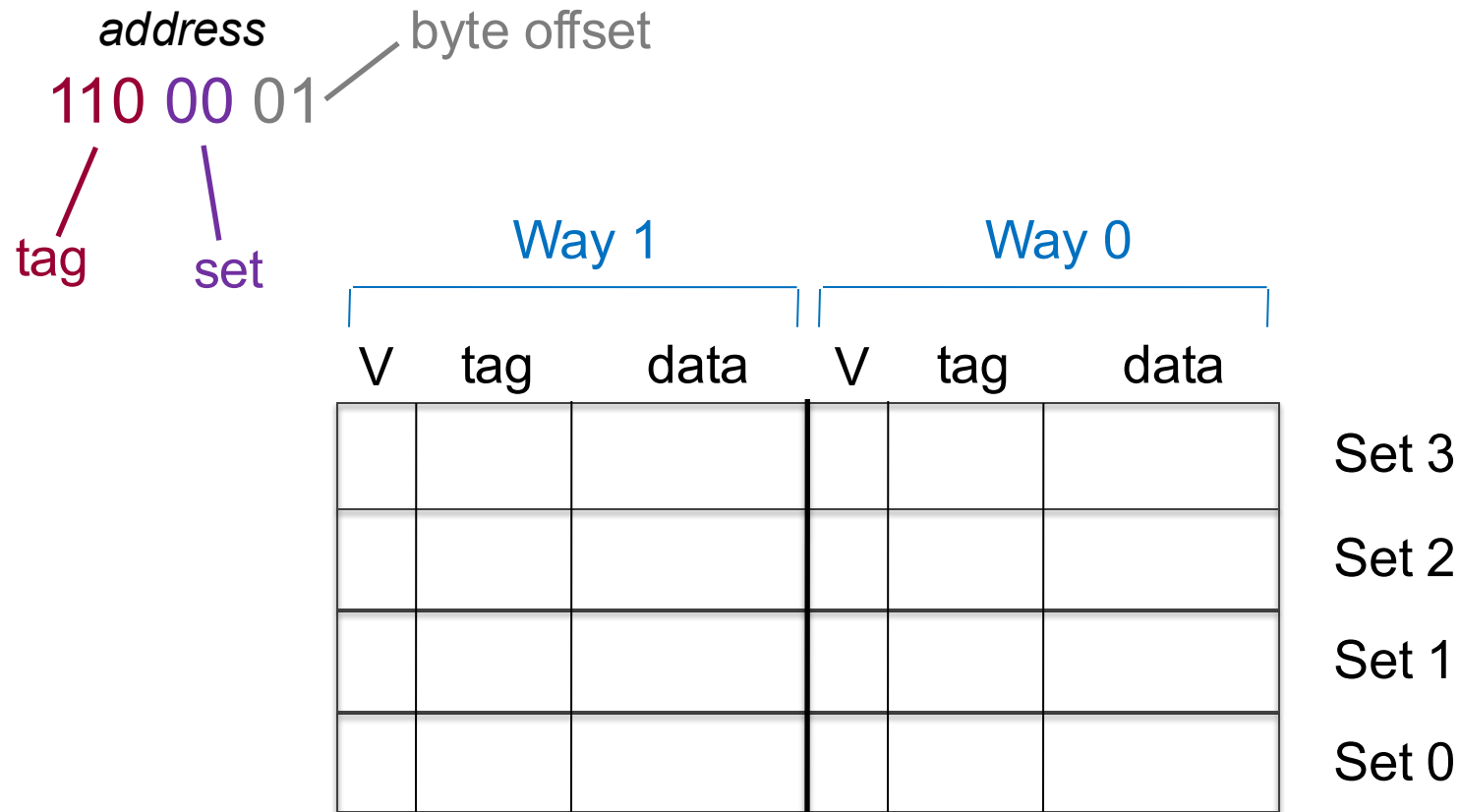
address

110 00 01

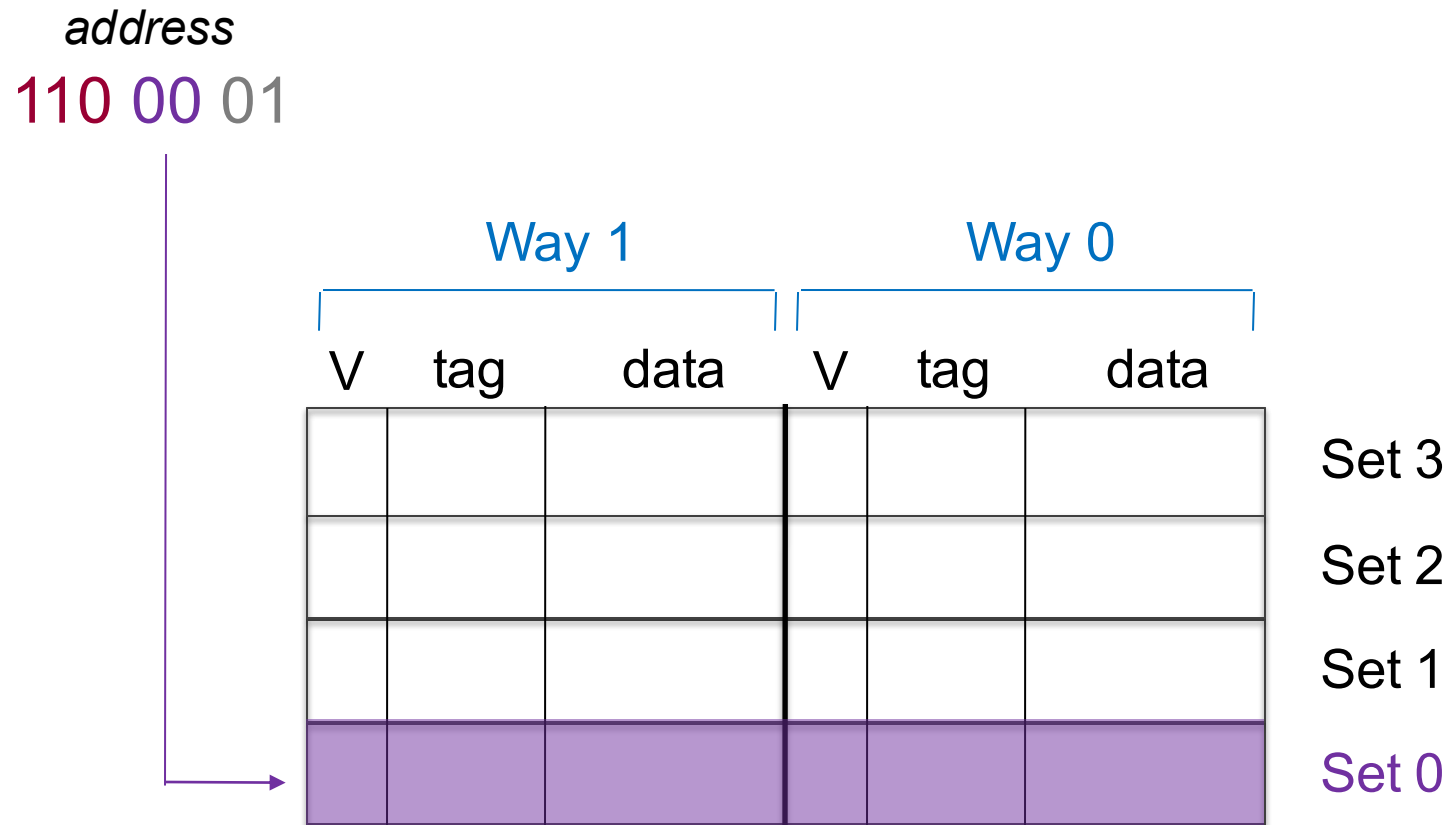
Suppose we
want to read
from memory

Way 1			Way 0			
V	tag	data	V	tag	data	
						Set 3
						Set 2
						Set 1
						Set 0

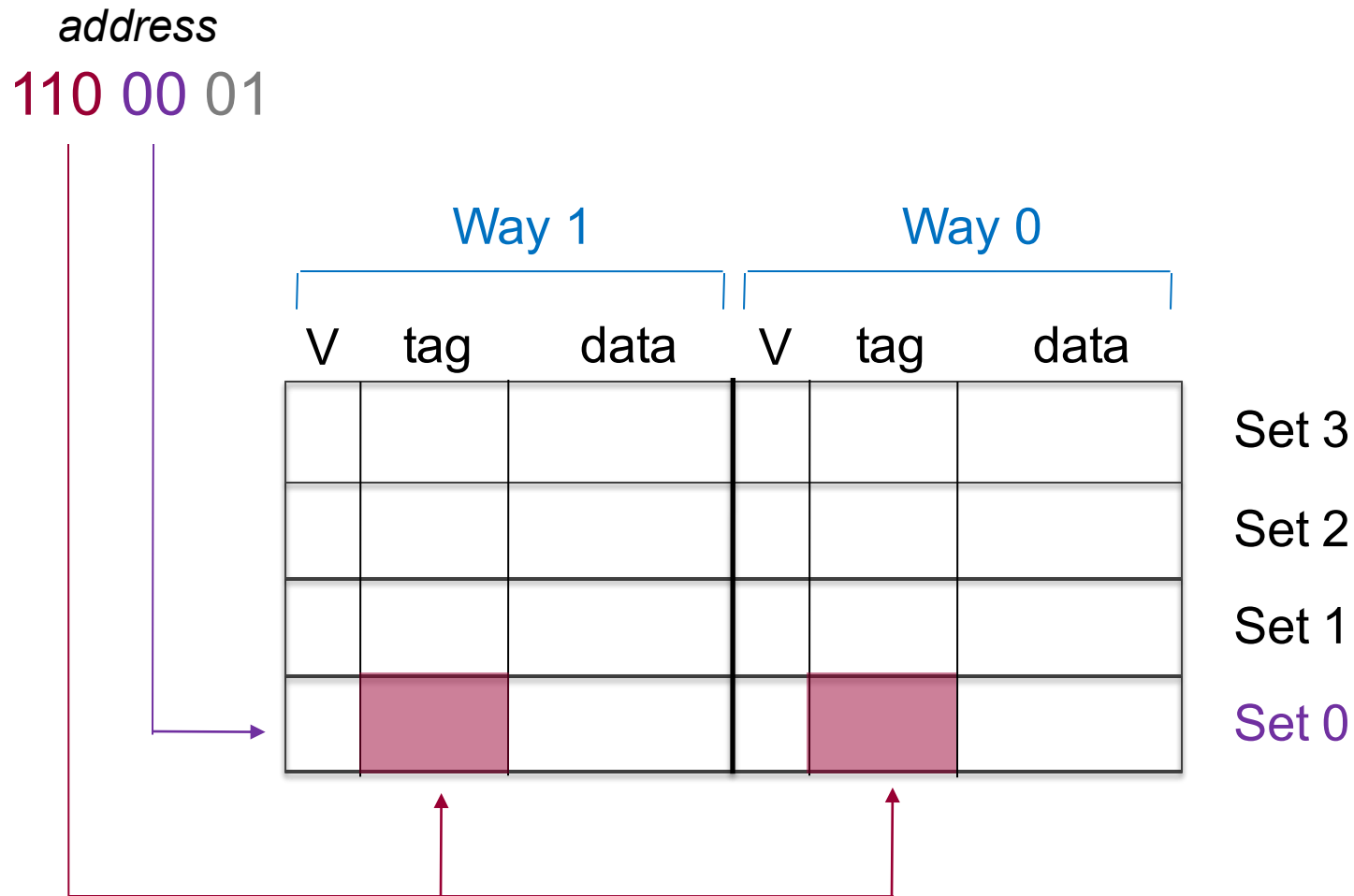
Interaction with memory: caches internals



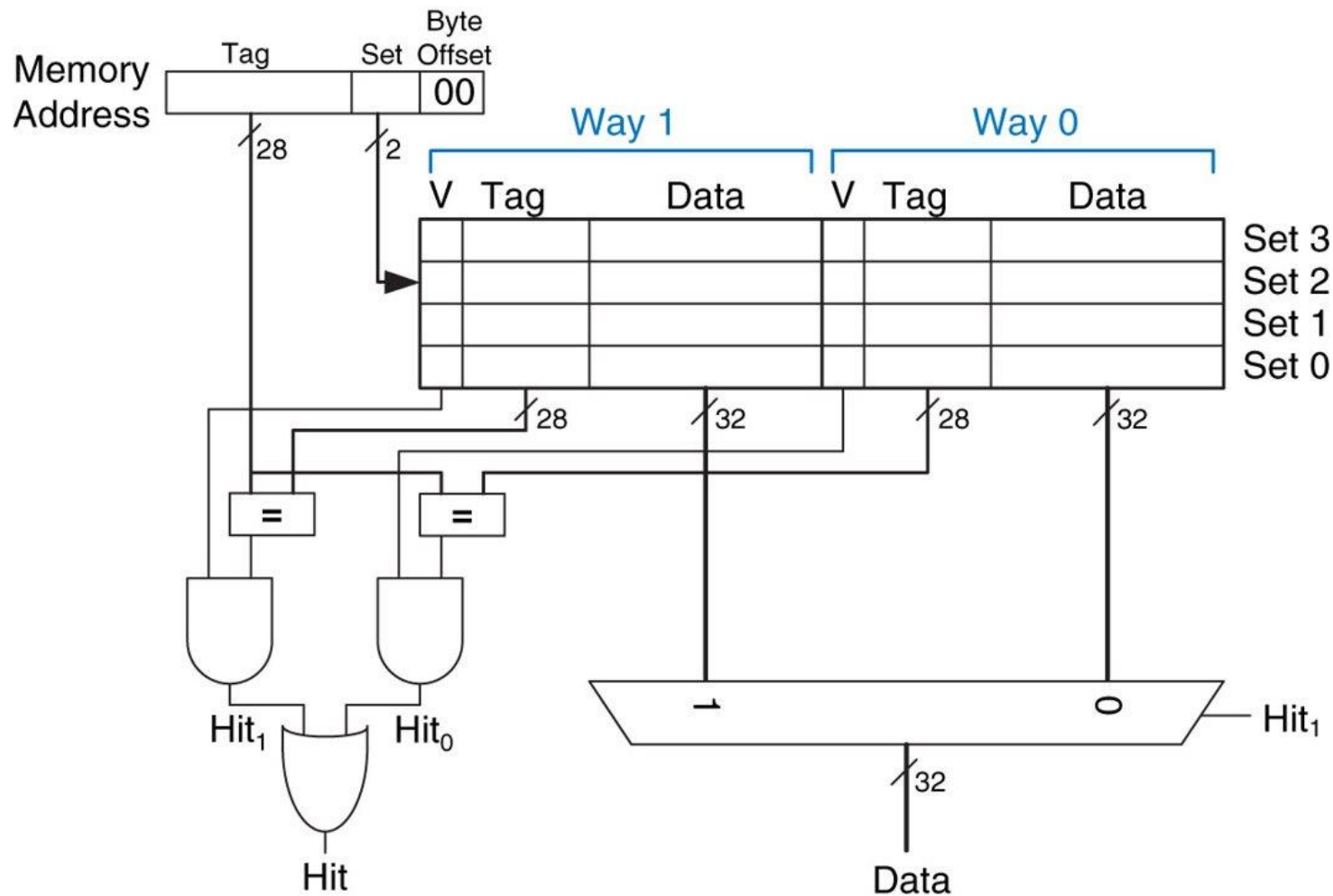
Interaction with memory: caches internals



Interaction with memory: caches internals



Interaction with memory: caches internals

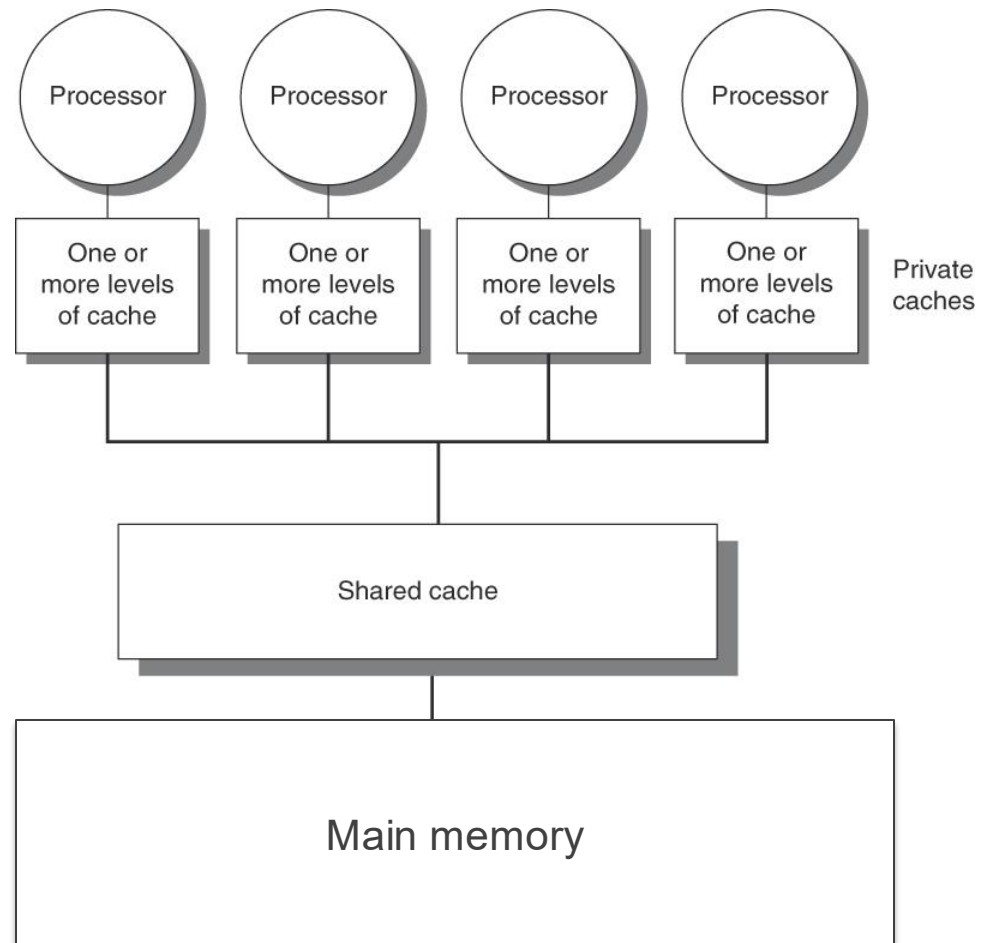


Interaction with memory: caches in practice

- Cache **dimensioning**
 - e.g., how many sets? How many blocks per set (i.e., ways)? What block size (i.e., data per way)?
- **Replacement strategy**, when need to store new block in an occupied cache slot
 - e.g., Least Recently Used vs random
- **Writing strategy**, when cache entries are modified
 - e.g., write-through vs write-back
- Also, **how many caches?**
 - e.g., three levels of caches, progressively bigger as they are farther away from the processor

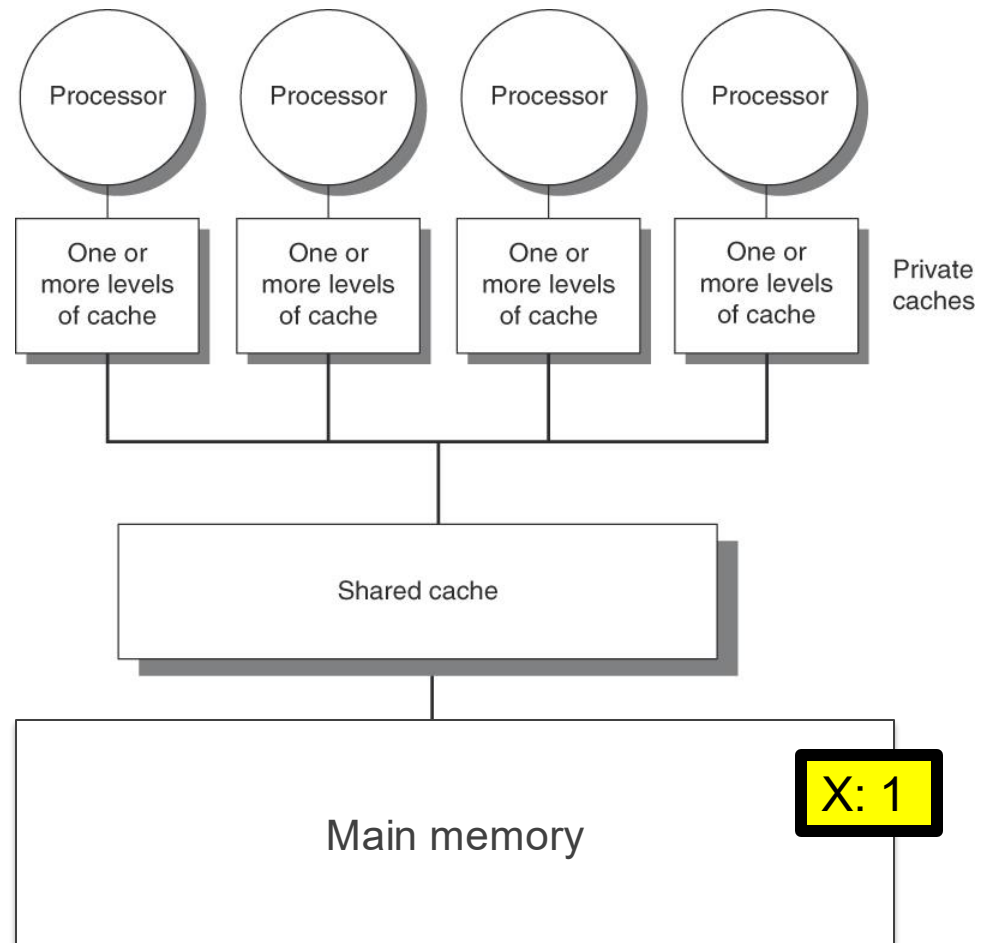
Interaction with memory: a step forward

- Suppose four processors



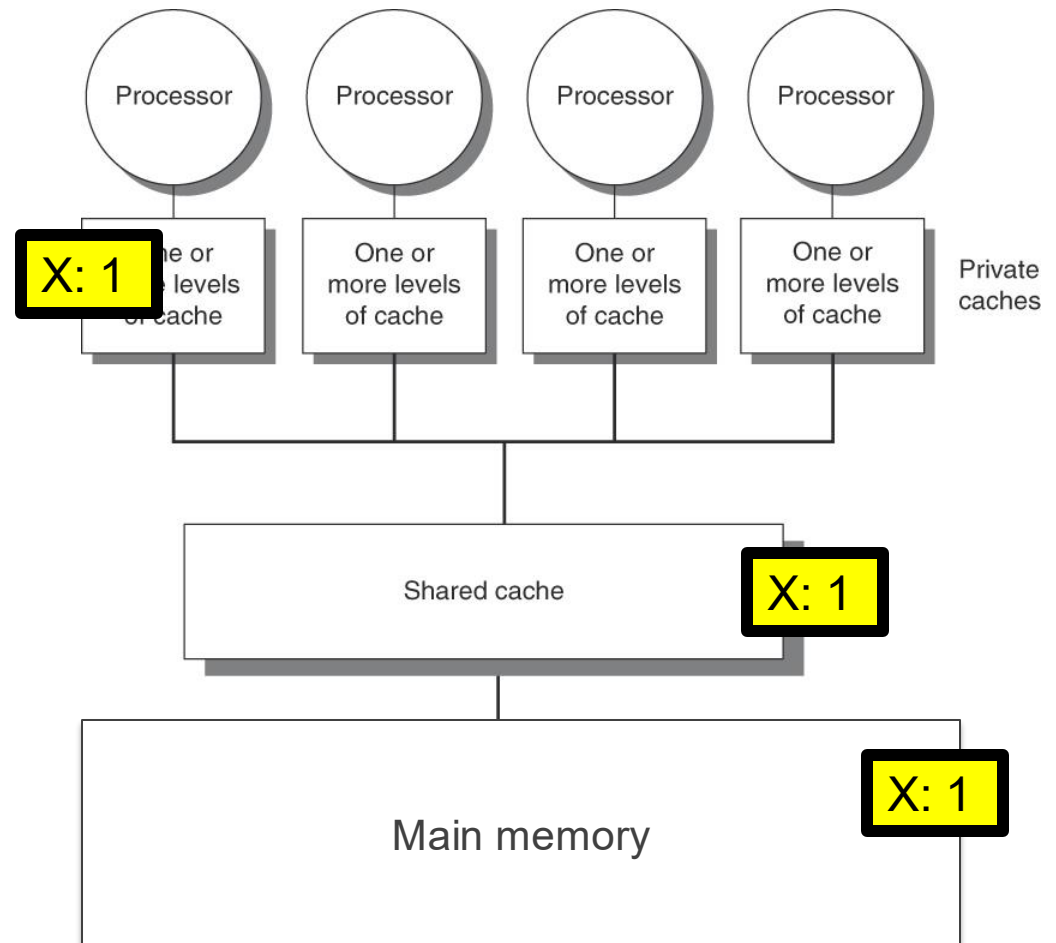
Interaction with memory: a step forward

- Suppose four processors and the following events
 - t0: main memory location X stores 1



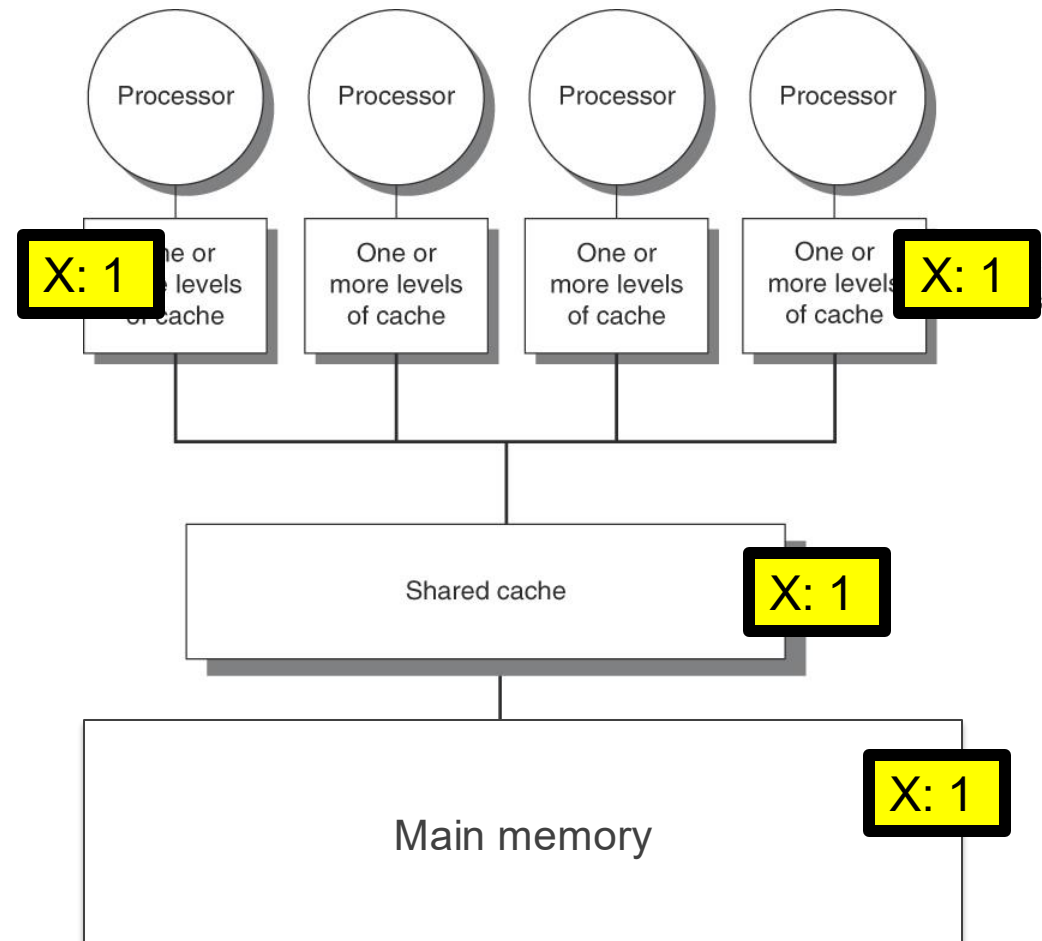
Interaction with memory: a step forward

- Suppose four processors and the following events
 - t0: main memory location X stores 1
 - t1: processor 1 caches X = 1



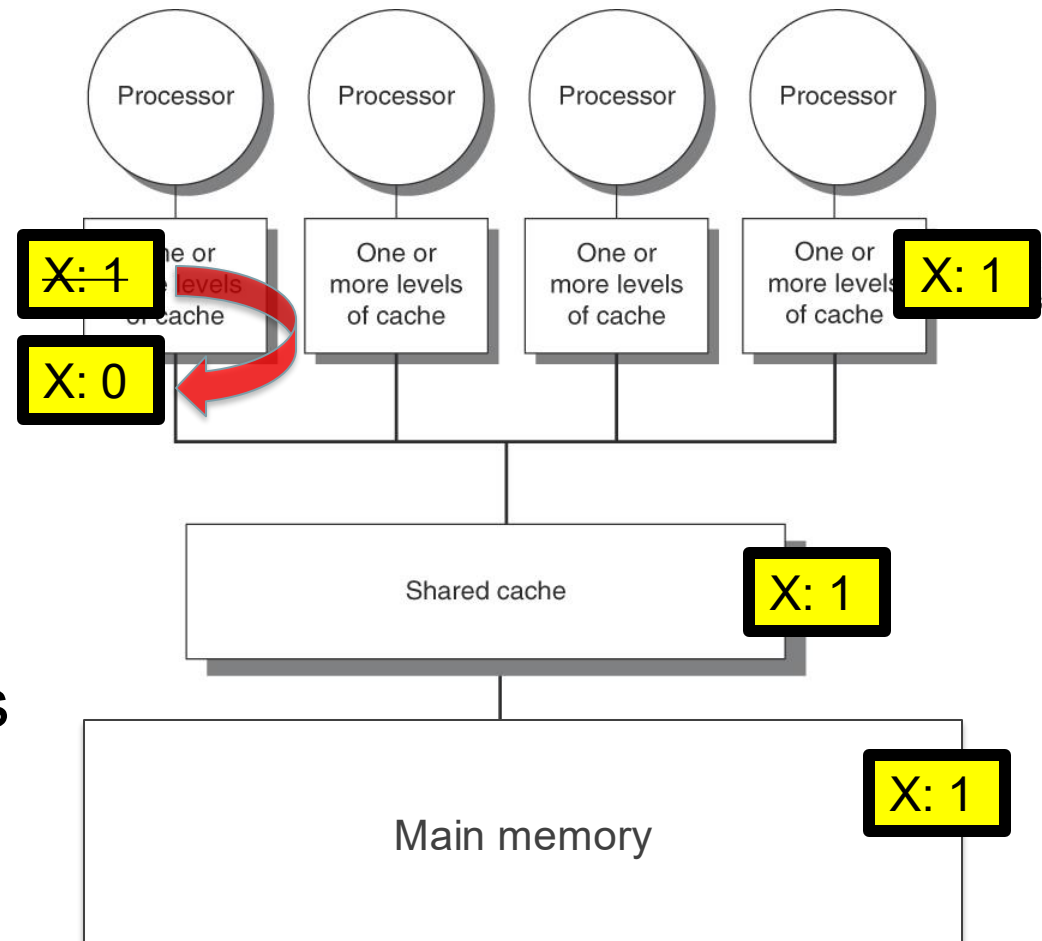
Interaction with memory: a step forward

- Suppose four processors and the following events
 - t0: main memory location X stores 1
 - t1: processor 1 caches X = 1
 - t2: processor 2 caches X = 1



Interaction with memory: a step forward

- Suppose four processors and the following events
 - t0: main memory location X stores 1
 - t1: processor 1 caches X = 1
 - t2: processor 2 caches X = 1
- What happens if X is modified in one cache at time t3?



Lessons (to be) learned

- Compilers and processors can **re-order operations** to improve execution speed
- Caching adds uncertainty on **where updated values are stored** at any point in time
 - ... and hence if updated values are **visible** everywhere in the program, at any time
- Lesson: **programmers can assume very little about the execution** of instructions in hardware
 - starting from next week, we will see how this affects software in high-level languages

Lessons (to be) learned

- Hardware concurrency is meant to improve performance (i.e., instruction throughput)
- Yet, **concurrency risks to affect correctness**
 - e.g., reading stale values from registers
- Lesson: **sometimes we must sacrifice performance** to ensure correctness
 - i.e., restricting the set of operations run in parallel
- Lesson: adding **synchronization points** is a general tool towards correctness of concurrent actions
 - e.g., **wait** for an operation to finish (e.g., set a register) before performing another (e.g., read that register)

Additional content

- Other types of pipelining hazards [P&H 4.5]
 - namely, structural and control ones
- Memory hierarchy [P&H 5.1-5.3, Moodle]
 - especially, details and role of caches

Optionally:

- Internals of MIPS processors [P&H 4, Moodle]
 - including more details on hardware components, as well as step-by-step execution of some instructions
- Advanced pipelining techniques [P&H 4, Moodle]
 - e.g., superscalar processor and dynamic scheduling